

DESIGN OF CACHE MEMORY

*Dhruv Jaimini (21BEC029)
Nirma University ,Ahmedabad
21bec029@nirmauni.ac.in*

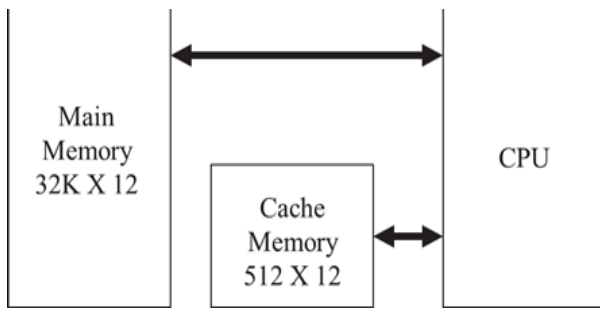
*Aditya Srivastava(21BEC005)
Nirma University,Ahmedabad
21bec005@nirmauni.ac.in*

Abstract— This project proposes improvements in cache management in conjunction with main memory; it aims to analyze the execution time differences between a direct-mapped and two-way set-associative-mapped cache system. Direct mapped is optimized in phase one by getting reference data into the initial memory blocks to reduce compulsory misses. LRU is included in the 2-way set associative cache to maximize the hit ratio among sets and minimize the miss ratio, in fact, particularly conflict misses. It is also mentioned in the paper that set associative mapping is proposed and a few possible developments in the area of multicore processing are indicated. ***Keywords: RAM, Miss Ratio, Hit Rate, Hit Ratio, Tag bits and Addresses, 2 way associative mapping, direct mapping.***

I. INTRODUCTION

Cache memory has an enormous influence on system performance because it has a very high speed, in the same order of magnitude, as that of the CPU elements. It functions on the idea of "reference locality," by which several memory references are localized. Those variables and active program instructions that are in the cache reduce the average memory access time and therefore reduce the overall program execution time. The principal goal for cache organisation is

to reduce the average access time for memory to a point nearly equal to the average cache access time. Cache memory mostly contains instructions and data; and it receives thousands of requests for memory compared to main memory. Computers come with many levels of cache memory, with the biggest and the slowest at the final level. Whenever a word needs fetching, the CPU looks into its cache first for the word's main address. A successful search is called a hit; otherwise, it is a miss. It triggers the retrieval of the data from the main memory, which is then stored in the cache for possible use in the future. Initial access failures and the possibility of address conflicts can be resolved in such a way that the hit ratio, being the fraction of the number of hits over the total number of accesses, is made to be as high as possible. As a buffer and temporary storage area, the cache memory is the middle ground between the processors and the system main memory; it is usually located in chips.

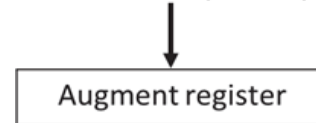


Recent studies have pointed out the fact that to improve system performance; internal factors of the multi-core system, like cache efficiency, are very important. Cache is closer to the microprocessor, and therefore better in performance than the main memory and the RAM. Synchronization of data transfer between the processor and main memory should be done for the processor to provide seamless data fetching. Although caching is more efficient than the use of RAM, it is more power consuming since it is an on-chip feature. With the ever-increasing speed of the processors, it becomes important, more than ever before, to increase the size of the cache memories to be able to cope with the ever-increasing gap between the CPU and memory speeds. In that case, to increase the cache speed, three such techniques of a victim cache, sub-blocks, memory bank are used in a simultaneous way where these technologies are trying to close down the speed gap between cache and main memory. The technologies are consecutively placed to enhance the speed and function of cache and already existing variables like cache mismatch, access rate, miss ratio are used to grade the performance of the cache after the implementation of the proposed method.

II. BACKGROUND

The article discusses various caching techniques, focusing on three main methods:- associative mapping, direct mapping, and set associative mapping.

CPU address (15 bits)



Address	Data
01000	3450
02777	6710
22345	1234

1. **Associative Mapping:** This technique employs associative memory for caching, offering flexibility and speed. It makes any accessible address in the cache memory to store any word from the main memory. When the CPU sends a 15-digit address to the argument register, the associative memory looks up the address. If the address is found in the cache, the corresponding 12-bits data is retrieved; otherwise, it's fetched from the main memory cell and stored in the cache respectively.

2. **Direct Mapping:** This method divides the processor's 15-bit address into an index part and an identifier field. The index field determines cache access, while the identifier field helps locate data in the main memory. Direct mapping requires main memory access, with cache words organized to store both data and associated tags.

3. **Set Associative Mapping:** This approach combines aspects of direct and associative mappings. Each cache word contains identification forms and actual data words, grouped in sets.

When the CPU accesses memory, it compares the tags in the CPU address with those in the cache, aiming to identify hits. This logical comparison resembles an associative memory search, enhancing hit ratio through increased array size and enabling additional identifiers to point to the same directory.

Overall, these techniques offer different approaches to cache organization, each with its advantages and trade-offs.

III. PROPOSED ARCHITECTURE WHICH IS BEING FOLLOWED :-

Since it has been underlined above that there exists a speed disparity between memory cache and main memory, a lot of research has to be done to address this. This section deals with an architectural way of addressing this problem. The architecture gives a systematic way to bring into one approach the limitations of previous techniques to improve the speed of cache. Memory will be increased by this step by step architecture, taking into account all techniques in a comprehensive manner, where every technique is helpful in improving the performance of the memory. In addition, different methods and methods have been investigated in the present paper to assess the usefulness of such technologies for enhancing performance.

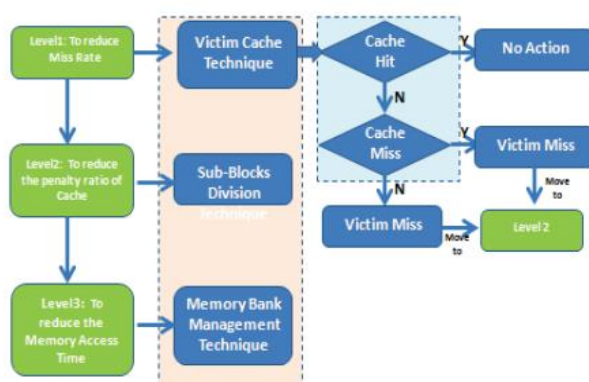


Fig. 5. Proposed architecture for improving Cache performance

Step 1: Reducing The Miss Rate using Victim Cache :-

A victim cache is a kind of hardware cache aimed at minimizing hit latencies in direct mapped caches and completely removing contention. Herein, the action of the victim cache with respect to the corresponding level of the hierarchy of the cache with respect to cache interactions is explained. If it is a victim cache hit and the cache misses, the victim cache replaces the cache block with the blocks that it contains. This will ensure that the block newly stored in the victim cache becomes the block recently accessed. Otherwise, in any case of a miss in the cache and the victim, a part from the next level is brought in the cache and the evicted part from the cache is placed into the victim cache.

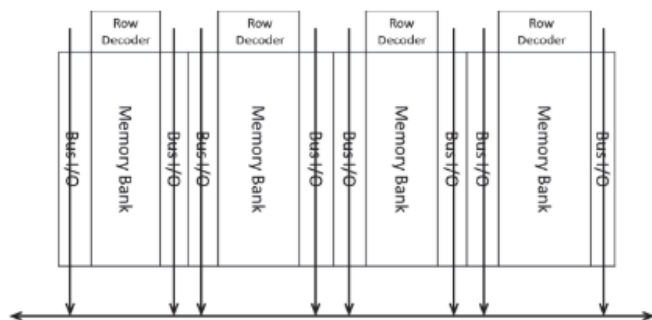
Step2: Reducing the Penalty Ratio of Cache Miss :-

Large tags consume space and therefore do not enhance cache performance as speed. One of the methods of making the tags efficient is the use of large blocks on the chip that will reduce the length of the tags and in the process reduce the number of cache misses. However, the entire block is transferred between the cache and the other components of the memory at a very high penalty. The blocks are sub-divided into smaller sub-blocks that are given a valid portion of their own to reduce this penalty. The tag is still valid for the entire block. In case of a miss, however, the relevant sub-block is read. This reduces the penalty, since the block is no longer regarded as the minimum transfer unit between the cache and memory.

Step3: Decreasing The Memory Access Time

Banks That Optimize Memory Involves Organize The time to reduce usage time Banks Optimize Memory that organizes memory locations on hard drive and manages logical storage held in computer hardware by memory controller. The banks

of associative memory are organized in a way that is almost the same as the associative memory and the way to memory banks are accessed sequentially through an array. If the bank that data item, $a(n)$ is stored is register (b), then the subsequent data item $a(n+1)$ is stored in register (b+1). By so doing, the cache is divided into several banks to avoid the bank cycle effects, and in the same case maintain constant processing by giving the chance of letting each bank recover in time before the next operation of storage or retrieval of data. Therefore, this calls for demand in different numbers of memory modules that are used to support the same size of the data bits as is illustrated in Figure 8.

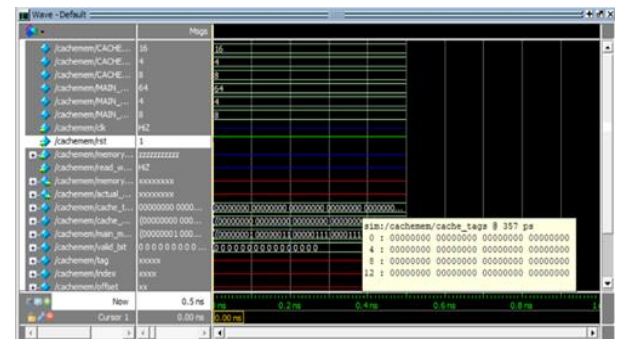


IV . CONCLUSION AND FUTURE SCOPE :-

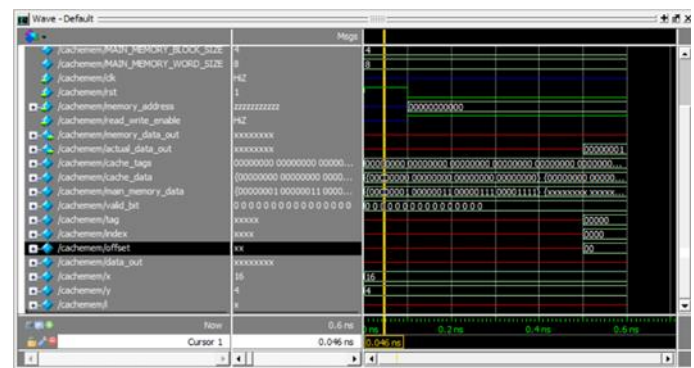
The project takes into consideration all the diverse techniques directed at increasing performance with the cache memory design, and particularly with the set-associative and direct mapping technique. Now, there are pros and cons to both kinds of mapping techniques. Another major finding of this research would be that the block size can be increased to improve the conflict miss rate but would utilize more cache space. However, the larger block sizes can raise miss penalty ratios, though at the same time they can lower hit latency and power. Larger caches usually offer slower access times and higher costs, whereas more associativity means faster access times with smaller cycle counts. Although caches lower miss rates, they do so at more cost than look-aside miss

caches. There are two conditions in which direct mapping applies: the reset condition, in which the whole cache first misses before data are remapped from the main memory, and set-associative mapping provides faster access time, which further reduces cycle count.

1. The reset condition in the Direct Mapping Technique is being used here in the case of which the complete cache Miss which is the compulsory miss in the starting of the memory and then the data is being again mapped into the cache through the main memory .



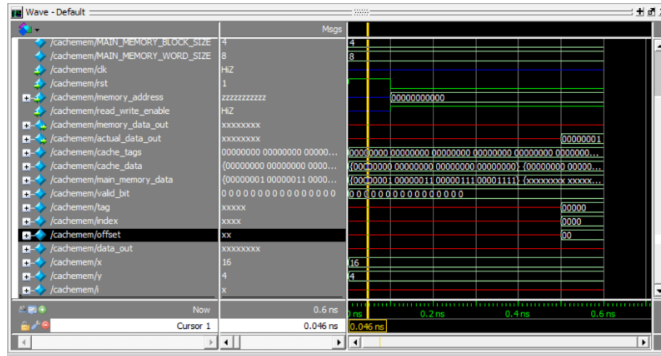
2. The Below give condition is of the Cache memory mapping from the main memory how the data in the cache is being loaded using the direct mapping is being clearly visible through the output waveforms.



3. The Set Associative memory Mapping is divided into a number of sets, each containing multiple cache lines and When data is fetched from or stored into the main memory, it is cached in one of the cache lines within a particular set. The mapping process involves determining which set a

particular memory address should be placed in.

The above result shows that how accurate the data is and it is how the waveform can be verified using the ModelSim stimulator and the above result shows the same .



V. REFERENCES :-

- [1] V. Chaplot, “Virtual Memory Benefits and Uses,” International Journal of Advance Research in Computer Science and Management Studies, vol. 4, no. 9, 2016.
- [2] C. Zhang, F. Vahid, and R. Lysecky, “A Self-Tuning Cache Architecture for

Embedded

Systems,” ACM Transactions on Embedded Computing Systems, vol. 3, no. 2, pp. 407-425, 2004.

[3] A. Agrawal and S. D. Pudar, “Column-associative Caches: a

Technique for Reducing the Miss Rate of Direct-mapped Caches,” in Proc. of the 20th Annual International Symposium on Computer Architecture, 1993, pp. 179-190. [4] S. Kumar and P. Singh, “An overview of modern cache memory and performance analysis of

replacement policies,” in Proc. of IEEE International Conference on Engineering and Technology (ICETECH), 2016, Coimbatore, India, page. 213-214.

[5] S. S. Oran and I. A. Amor, “Implementation of LRU Replacement Policy for Reconfigurable Cache Memory Using FPGA,” in Proc. of the International Conference on Advanced Science and Engineering, 2018, page 13-18.

Annexure

1) Code for direct mapping cache:

```
module cachemem(
    input clk,
    input rst,
    input [10:0] memory_address,
    input read_enable,
    input write_enable,
    output reg [7:0] memory_data_out
);

// Parameters
parameter CACHE_SIZE = 16; // Reduced cache
size to 16 lines
parameter CACHE_BLOCK_SIZE = 4; // Number
of words per cache line
parameter CACHE_WORD_SIZE = 8; // Size of
each word in the cache
parameter MAIN_MEMORY_SIZE = 64; //
Number of lines in main memory
parameter MAIN_MEMORY_BLOCK_SIZE = 4;
// Number of words per main memory line
parameter MAIN_MEMORY_WORD_SIZE = 8; //
Size of each word in main memory

// Data structures
reg [7:0] cache_tags [0:CACHE_SIZE-1];
reg [CACHE_WORD_SIZE-1:0] cache_data
[0:CACHE_SIZE-1][0:CACHE_BLOCK_SIZE-1];
// Cache data
reg [MAIN_MEMORY_WORD_SIZE-1:0]
main_memory_data [0:MAIN_MEMORY_SIZE-
1][0:MAIN_MEMORY_BLOCK_SIZE-1]; // Main
memory data
reg [0:0] valid_bit [0:CACHE_SIZE-1];

// Internal signals
reg [4:0] tag;
reg [3:0] index;
reg [1:0] offset;
reg [7:0] data_out;

// Cache control logic
// Cache control logic
always @(posedge clk or posedge rst) begin
    if (rst) begin
        tag = memory_address[10:6];
        index = memory_address[5:2];
        offset = memory_address[1:0];

        for (index = 0; index < CACHE_SIZE; index =
index + 1) begin
            cache_tags[index] <= 8'b0; // Reset cache
tags to all zeros
            valid_bit[index] <= 1'b0; // Reset valid bit to 0
            for (offset = 0; offset <
CACHE_BLOCK_SIZE; offset = offset + 1) begin
                cache_data[index][offset] <=
{CACHE_WORD_SIZE{8'b0}}; // Reset cache
data to all zeros
            end
        end
    end else if (read_enable | write_enable) begin
        // Cache operation logic
        if (valid_bit[index] == 1 && cache_tags[index]
== tag) begin
            data_out = cache_data[index][offset];
        end else begin
            data_out =
{MAIN_MEMORY_WORD_SIZE{8'bX}}; //
Default data for cache miss
            for (offset = 0; offset <
MAIN_MEMORY_BLOCK_SIZE; offset = offset +
1) begin
                cache_data[index][offset] <=
main_memory_data[index][offset];
            end
            cache_tags[index] = tag;
            valid_bit[index] = 1'b1;
        end
    end
end

// Interface with main memory
always @(*) begin
    memory_data_out =
main_memory_data[tag][offset];
end

endmodule

// Reset cache content to 0
```

```
for (index = 0; index < CACHE_SIZE; index =
index + 1) begin
    cache_tags[index] <= 8'b0; // Reset cache
tags to all zeros
    valid_bit[index] <= 1'b0; // Reset valid bit to 0
    for (offset = 0; offset <
CACHE_BLOCK_SIZE; offset = offset + 1) begin
        cache_data[index][offset] <=
{CACHE_WORD_SIZE{8'b0}}; // Reset cache
data to all zeros
    end
end
end else if (read_enable | write_enable) begin
    // Cache operation logic
    if (valid_bit[index] == 1 && cache_tags[index]
== tag) begin
        data_out = cache_data[index][offset];
    end else begin
        data_out =
{MAIN_MEMORY_WORD_SIZE{8'bX}}; //
Default data for cache miss
        for (offset = 0; offset <
MAIN_MEMORY_BLOCK_SIZE; offset = offset +
1) begin
            cache_data[index][offset] <=
main_memory_data[index][offset];
        end
        cache_tags[index] = tag;
        valid_bit[index] = 1'b1;
    end
end
end

// Interface with main memory
always @(*) begin
    memory_data_out =
main_memory_data[tag][offset];
end

endmodule
```

2) Code for 2 way set associative mapping:

```

module ca_sa(
    input clk,           // External clock
    input [7:0] data_in, // Input to the cache in byte
    input mem_read, mem_write, // Used to indicate if
    read or write operation is performed
    input [15:0] address_bus, // 16-bit address bus for
    accessing bytes
    input [15:0] mem_data, // Data present in the
    cache memory
    output reg [7:0] data_out, // Output of the data
    output reg cache_hit, cache_miss // Indicates if
    there is a hit or a miss
);

reg [29:0] way1 [7:0]; // 2-way cache with each
line having 2 ways or sets
reg [29:0] way2 [7:0];
reg [11:0] tag_addr; // Tag present in the bytes
of address bus
reg [11:0] tag_1, tag_2; // Tag of way/set 1 and 2
reg validbit_1, validbit_2; // Valid bit present in the
2-way sets
reg offset; // Offset bit determines which
way will be used out of the 2 ways present
reg [2:0] index; // Determines the line
number in the cache memory
reg lrubit_1, lrubit_2; // LRU bit used to implement
the replacement policy of Pseudo LRU
always @(posedge clk) begin
    index[2:0] <= address_bus[3:1]; // Extracting index
bits from the address bus
    offset <= address_bus[0]; // Extracting the
offset bit from the address bus
    tag_addr <= address_bus[15:4]; // Extracting the
tag bits from the address bus
    validbit_1 <= way1[index][29]; // Extracting valid
bit from way 1 in cache
    validbit_2 <= way2[index][29]; // Extracting valid
bit from way 2 in cache
    tag_1[11:0] <= way1[index][28:17]; // Extracting tag
bits from way 1 in cache
    tag_2[11:0] <= way2[index][28:17]; // Extracting tag
bits from way 2 in cache
    lrubit_1 <= way1[index][0]; // Extracting LRU bit
from way 1 to determine least recently used
    lrubit_2 <= way2[index][0]; // Extracting LRU bit
from way 2 to determine least recently used

    // Read operation
    if (mem_read == 1'b1) begin // Check if read is
    performed
        if (validbit_1 == 1'b1) begin // Check valid bit for
        way 1
            if (tag_addr == tag_1) begin // Check if tag bits
            of way 1 match

```

```

            if (offset == 1'b0) begin // If offset is 0, read
            from LSB byte of data line
                data_out <= way1[index][8:1]; // Lowest
            significant byte
                cache_hit <= 1'b1;
                cache_miss <= 1'b0; // Cache hit
                lrubit_1 <= 1'b1;
                lrubit_2 <= 1'b0; // Set LRU bits
                way1[index][0] <= lrubit_1; // Update LRU
            bits
                way2[index][0] <= lrubit_2;
            end
            else begin // If offset is 1, read from second byte
            of data line
                data_out <= way1[index][16:9]; // Most
            significant byte of way 1
                cache_hit <= 1'b1;
                cache_miss <= 1'b0; // Cache hit
                lrubit_1 <= 1'b1;
                lrubit_2 <= 1'b0; // Set LRU bits
                way1[index][0] <= lrubit_1; // Update LRU
            bits
                way2[index][0] <= lrubit_2;
            end
            end
            // Check if tag bits of way 2 match and valid bit is
            also set
            else if (tag_addr[11:0] == tag_2[11:0] &&
            validbit_2 == 1'b1) begin
                if (offset == 1'b0) begin // Offset 0, read from
                first byte of way 2
                    data_out <= way2[index][8:1];
                    cache_hit <= 1'b1;
                    cache_miss <= 1'b0; // Cache hit
                    lrubit_1 <= 1'b0;
                    lrubit_2 <= 1'b1; // Set LRU bits
                    way1[index][0] <= lrubit_1; // Update LRU
                bits
                    way2[index][0] <= lrubit_2;
                end
                else begin // Offset 1, read from second byte of
                way 2
                    data_out <= way2[index][16:9];
                    cache_hit <= 1'b1;
                    cache_miss <= 1'b0; // Cache hit
                    lrubit_1 <= 1'b0;
                    lrubit_2 <= 1'b1; // Set LRU bits
                    way1[index][0] <= lrubit_1; // Update LRU
                bits
                    way2[index][0] <= lrubit_2;
                end
                end
            else begin
                cache_miss <= 1'b1; // Cache miss if tag bits do
                not match
                $display("There is a miss and the data
                requested will be read from the main memory to
                cache");
            end

```

```

end
end

// Write operation
if (mem_write == 1'b1) begin
  if (validbit_1 == 1'b1) begin
    if (tag_addr[11:0] == tag_1[11:0]) begin
      if (offset == 1'b0) begin
        way1[index][8:1] <= data_in;
        cache_hit <= 1'b1;
        cache_miss <= 1'b0;
        lrubit_1 <= 1'b1;
        lrubit_2 <= 1'b0;
        way1[index][0] <= lrubit_1;
        way2[index][0] <= lrubit_2;
      end
    else begin
      way1[index][16:9] <= data_in;
      cache_hit <= 1'b1;
      cache_miss <= 1'b0;
      lrubit_1 <= 1'b1;
      lrubit_2 <= 1'b0;
      way1[index][0] <= lrubit_1;
      way2[index][0] <= lrubit_2;
    end
  end
else if (tag_addr[11:0] == tag_2[11:0]) begin
  if (offset == 1'b0) begin
    way2[index][8:1] <= data_in;
    cache_hit <= 1'b1;
    cache_miss <= 1'b0;
    lrubit_1 <= 1'b0;
    lrubit_2 <= 1'b1;
    way1[index][0] <= lrubit_1;
    way2[index][0] <= lrubit_2;
  end
else begin
    way2[index][16:9] <= data_in;
    cache_hit <= 1'b1;
    cache_miss <= 1'b0;
    lrubit_1 <= 1'b0;
    lrubit_2 <= 1'b1;
    way1[index][0] <= lrubit_1;
    way2[index][0] <= lrubit_2;
  end
end
else begin
  cache_miss <= 1'b1;
  $display("There is a miss");
end
end
end
end
endmodule

```